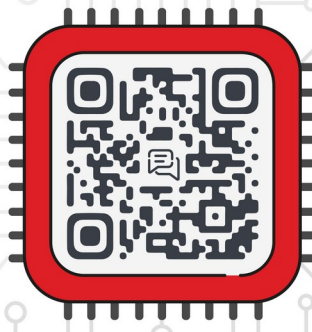




نظري

10

11

1320 sp

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

Advanced
Application Security

د. غسان شذور



RB Informatics; محتوى مجاني غير مخصص للبيع التجاري
5/1/2025

برمجة تطبيقات شبكية

السلام عليكم ورحمة الله وبركاته

في هذه المحاضرة سنتعلم :

Spring Security: تحليل البنية المعتمدة على المرشحات (Filter-based Architecture) في Spring Security وكيفية عمل سلسلة المرشحات (Filter Chain) ومعالجة الطلبات الواردة والصادرة لضمان الأمن. تصميم أنظمة مصادقة تعتمد على رموز JWT: مع إدارة كاملة لدورة حياة الرمز، بالإضافة إلى كيفية استخدامها للمصادقة والترخيص، وتقليل المخاطر. تنفيذ مرشحات أمنية مخصصة (Custom Security Filters): لتوفير حماية مخصصة لتلائم احتياجات التطبيق دون المساس بالتكوينات الافتراضية، مما يوفر حماية مرنة ومهنية. إنشاء بروتوكولات TLS/HTTPS باستخدام معايير التحديث الحديثة: فهم أهمية أمن طبقة النقل (Transport Layer Security - TLS) وكيفية إنشائها بشكل صحيح في تطبيقات Spring Boot لضمان سرية وسلامة الاتصالات عبر الشبكة.

Spring Security Architecture

تُعد Spring Security مكتبة شاملة وقوية توفر حلاً أمنياً متكاملًا لتطبيقات Spring. تعتمد فلسفتها الأساسية على مبدأ "Security by default, customizable when needed"، مما يعني أنه بمجرد إضافة الاعتماد spring-boot-starter-security إلى مشروع Spring Boot، يتم تفعيل حماية شاملة افتراضية دون الحاجة إلى تكوين يدوي معقد.

تشمل هذه الحماية الافتراضية:

1. تأمين جميع نقاط النهاية (Endpoints) بشكل افتراضي: يتم حماية جميع مسارات التطبيق (Paths)، مما يمنع الوصول إلى أي جزء من التطبيق دون تسجيل دخول.

2. إنشاء صفحة تسجيل دخول افتراضية: توفر Spring Security صفحة جاهزة لتسجيل الدخول للمستخدمين، مما يسهل عملية المصادقة الأولية.
3. دعم المصادقة القائمة على النماذج (Form-based Authentication): يتيح للمستخدمين تسجيل الدخول باستخدام نموذج HTML تقليدي.
4. توفير نقطة نهائية لتسجيل الخروج: تدعم عملية تسجيل الخروج الآمنة، مما يضمن إنهاء استخدام المستخدم بعد تسجيل خروجه.
5. توليد كلمة مرور عشوائية في وحدة التحكم (Console): يتم إنشاء كلمة مرور مؤقتة يمكن استخدامها للوصول إلى التطبيق المحلي.
6. تفعيل رؤوس الأمن الأساسية (Basic Security Headers): تُضاف رؤوس HTTP أمنية مثل X-Content-Type-Options و X-XSS-Protection و X-Frame-Options للحماية من هجمات XSS و Clickjacking.
7. تفعيل الحماية من تزيف الطلب عبر المواقع (CSRF Protection): يتم تفعيل هذه الحماية افتراضياً للعمليات التي تُغير حالة التطبيق (مثل POST, PUT, DELETE) لمنع هجمات CSRF.

سلسلة مرشحات الأمن (The Security Filter Chain)

تُعد قلب Spring Security؛ سلسلة من المرشحات تعالج كل طلب HTTP قبل الوصول إلى Controllers. تفحص وتعديل الطلبات حسب سياسات الأمن. ترتيب المرشحات حاسم لتطبيق قواعد الأمن بفعالية. تتكون السلسلة من عدد من المرشحات الرئيسية، كل منها مسؤول عن جانب معين من الأمن:

- مرشح المصادقة (Authentication Filter): يتحقق من هوية المستخدم عبر بيانات الاعتماد (اسم مستخدم، كلمة مرور). إذا نجح، ينشئ كائناً للمستخدم المصادق عليه.
- مرشح الترخيص (Authorization Filter): بعد المصادقة، يتحقق من صلاحيات المستخدم (roles/authorities) للوصول إلى المورد أو تنفيذ الإجراء المطلوب.
- مرشح حماية (CSRF Protection Filter): يمنع هجمات CSRF بإضافة رمز مميز فريد للطلبات التي تغير الحالة، ويرفض الطلب إذا كان الرمز مفقوداً أو غير صحيح.
- مرشح CORS (CORS Filter): يدير سياسة Cross-Origin Resource Sharing، محدداً النطاقات المسموحة بالوصول لمنع الطلبات cross-origin غير المرغوبة.
- إدارة الجلسة (Session Management): يتحكم في تخزين بيانات الجلسة؛ يدعم جلسات خادم أو stateless (مثل JWT) كخيار مفضل.

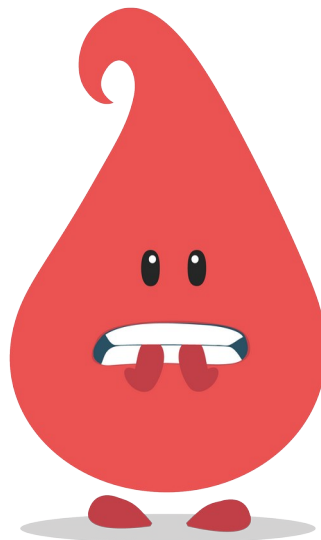
تُمكن قوة هذه البنية من إضافة مرشحات مخصصة أو تعديل تسلسل المرشحات الافتراضية لتلبية متطلبات أمنية محددة.

الأمان القائم على الرموز: تصميم وتنفيذ JWT

تُعد رموز (JWT) JSON Web Token حلاً حديثاً وفعالاً للمصادقة والترخيص في تطبيقات الويب الحديثة، خاصة في بيئة الـ Microservices والـ Stateless Applications. تقدم JWT بديلاً متقدماً للـ Traditional Sessions التي كانت تُستخدم بشكل شائع.

JWT (Traditional Session vs JWT) ↩

JWT Based-Session	Traditional Based-Session	وجه المقارنة
رمز ذاتي الاحتواء يُرسل مع كل طلب؛ لا تخزين على الخادم.	تعتمد على كوكيز لمعرفة الجلسة؛ البيانات على الخادم.	آلية العمل
التحقق من التوقيع الرقمي فقط؛ بدون استدعاءات DB.	الخادم يبحث في قاعدة البيانات/الذاكرة كل مرة.	التحقق
غير حالة (Stateless): قابلة للتوسع العالي؛ مثالية للـ Microservices.	حالة (Stateful): تستهلك موارد الخادم؛ أقل قابلية للتوسع.	الحالة
الرمز صالح حتى انتهاء صلاحيته؛ إبطال معقد.	عرضة لهجمات CSRF بدون حماية.	نقاط الضعف



يُكون رمز JWT من ثلاثة أجزاء مفصولة بنقاط (.): Header.Payload.Signature

1. الرأس (Header): يحتوي عادةً على نوع الرمز (مثل JWT) وخوارزمية التشفير (Algorithm) المستخدمة لتوقيع الرمز (مثل HS256 أو RS256).
2. الحمولة (Payload): تحتوي على المطالبات (Claims) وهي بيانات حول الكيان (عادةً المستخدم) وإضافات أخرى. يمكن أن تكون المطالبات مسجلة (Registered Claims) مثل iss (المصدر)، sub (الموضوع)، exp (تاريخ الانتهاء)، أو مطالبات عامة (Public Claims)، أو مطالبات خاصة (Private Claims) التي تحددها التطبيقات. هذه المطالبات تحمل معلومات المستخدم والصلاحيات.
3. التوقيع (Signature): يُستخدم للتحقق من أن الرمز لم يتم تعديله أثناء النقل، وللتأكد من أنه من أنشأ الرمز هو المصدر. يتم إنشاء التوقيع عن طريق أخذ الرأس والحمولة، ودمجهما مع مفتاح سري (Secret Key) باستخدام خوارزمية التشفير المحددة في الرأس.

ملاحظة مهمة: رمز JWT غير مشفر افتراضياً (Not Encrypted)، وهو موقع فقط (Signed). هذا يعني أن أي شخص يمكنه قراءة محتويات الرأس والحمولة (بعد فك تشفير Base64url)، ولكن لا يمكنه تعديلها دون إلغاء التوقيع. لذلك يجب عدم وضع بيانات حساسة جداً (Highly Sensitive Data) داخل الحمولة.

Token Lifecycle Management

تُعد إدارة دورة حياة رموز JWT أمراً غاية في الأهمية لضمان الأمان والفعالية. تعتمد الاستراتيجيات التقليدية عادةً على استخدام نوعين من الرموز:

رمز الوصول (Access Token): هذا الرمز قصير المدى (عادةً بين 5 إلى 15 دقيقة) يُستخدم للوصول إلى الموارد المحمية (Protected Resources). الهدف من قصره هو تقليل نافذة التعرض (Exposure Window) للمخاطر في حال تم اختراقه. عند انتهاء صلاحيته، يجب على العميل الحصول على رمز وصول جديد.

رمز التحديث (Refresh Token): هذا الرمز طويل المدى (عادةً بين 7 إلى 30 يوماً) يُستخدم للحصول على رمز وصول جديد عند انتهاء صلاحية رمز الوصول الحالي. يُخزن عادةً في قاعدة بيانات آمنة على الخادم. يُسمح بإلغاء رمز التحديث (Revocation) إذا تم اختراقه، مما يمنع المهاجم من الحصول على رموز وصول جديدة. كما أن حملته على كل طلب لا يتطلب استعلاماً لقاعدة البيانات (Database Load).

تُحقق هذه الاستراتيجيات توازناً بين الأمان والحياة. من ناحية، يُقلل من المخاطر المرتبطة باختراق رمز الوصول من خلال قصر مدته. من ناحية أخرى، يحافظ على خاصية "عدم الحالة" (Statelessness) للنظام مع جعله أكثر قابلية للتوسع والأداء.

Custom Security Filter Development

على الرغم من أن Spring Security توفر مجموعة واسعة من المرشحات الجاهزة (Out-of-the-box filters) التي تغطي معظم سيناريوهات الأمان الشائعة، إلا أنه هناك حالات تتطلب بناء مرشحات أمنية مخصصة (Custom Security Filters) لتلبية متطلبات أمنية فريدة أو لدعم نطاق أمني خاص بالتطبيق. تُعد القدرة على تطوير هذه المرشحات المخصصة ميزة قوية لـ Spring Security حيث تتيح للمطورين تخصيص سلسل الأمان بشكل كامل.

متى يُفضل بناء مرشحات مخصصة

- API Rate Limiting: تقييد عدد الطلبات من IP أو مستخدم معين خلال فترة زمنية لمنع هجمات Brute Force أو DDoS، مع رفض الطلبات الزائدة.
- Request Signing: توقيع الطلبات بمفتاح سري عبر HMAC للتحقق من سلامة الطلب وأصالته المصدر قبل المعالجة.
- IP Whitelisting: السماح فقط لعناوين IP أو شبكات محددة بالوصول إلى لوحات الإدارة أو APIs داخلية.
- Header Validation: التحقق من وجود رؤوس HTTP مطلوبة مثل X-API-Key لضمان صحة الطلبات.
- Audit Logging: تسجيل جميع الطلبات الأمنية (IP، مسار، نتائج المصادقة) في نظام تدقيق مركزي.
- Data Masking: إخفاء أو إزالة البيانات الحساسة من الاستجابات وسجلات التدقيق لمنع التسريب.

Custom Filter Implementation Pattern

عند بناء مرشح أمني مخصص في Spring Security، يتم عادةً اتباع نمط معين. يُفضل الوراثة من `OncePerRequestFilter` حيث يضمن تنفيذ المرشح مرة واحدة فقط لكل طلب HTTP حتى لو كان هناك إعادة توجيهه داخلية (Internal Forward/Redirect). يتضمن تنفيذ الخطوات التالية:

1. استخراج والتحقق من بيانات الأمان المخصصة: يستخرج المرشح بيانات أمنية مخصصة من طلب HTTP (مثل مفتاح API من رأس Authorization أو توقيع من X-Signature)، ثم يتحقق من صحتها.
2. إجراء الفحص الأمني: ينفذ المرشح الفحص الأمني (مثل توقيع غير صحيح أو مفتاح API مفقود/غير صالح). إذا فشل، يرفض الطلب برمز HTTP مناسب (401 Unauthorized أو 403 Forbidden) ويوقف سلسلة المرشحات.
3. متابعة سلسلة المرشحات: إذا نجح الفحص، يمرر الطلب للمرشح التالي عبر `filterChain.doFilter(request, response)`، مما يستمر معالجة الطلب بباقي مرشحات Spring Security ثم المتحكمات.
4. معالجة لاحقة اختيارية: بعد معالجة الطلب كاملاً، يمكن إجراء معالجة إضافية مثل تسجيل التدقيق أو تعديل الاستجابات قبل إرسالها للعميل.

إنشاء أمن طبقة النقل (Transport Layer Security - TLS/HTTPS)

يُعد أمن طبقة النقل (TLS) الذي كان يُعرف سابقاً باسم Secure Socket Layer (SSL)، مكوناً أساسياً لأي تطبيق ويب حديث. تُعد مشكلة الأمان الأساسية في الاتصالات الشبكية هي أن البيانات تُنقل عادةً بنص واضح (Plaintext) عبر بروتوكولات مثل HTTP، مما يجعلها عرضة للتصنّت (Eavesdropping) والتلاعب (Tampering) من قبل المهاجمين.

أفضل الممارسات الحديثة لـ TLS

لضمان أعلى درجات الأمان، يجب اتباع أفضل الممارسات عند تكوين TLS:

- تعطيل البروتوكولات القديمة: إيقاف TLS 1.0, TLS 1.1, SSLv2, SSLv3. استخدام TLS 1.2 كحد أدنى و TLS 1.3 مفضل لتجنب الثغرات الأمنية المعروفة.
- تفعيل البروتوكولات الحديثة: TLS 1.2 (حد أدنى) و TLS 1.3 (مفضل). يحسن TLS 1.3 الأداء والأمان بتقليل زمن المصافحة وإزالة الخوارزميات الضعيفة.
- أطقم تشفير قوية: تفضيل AEAD مثل AES-GCM و ChaCha20-Poly1305 للتشفير المصادق والنزاهة. تجنب CBC mode و 3DES لضعفهما.
- استخدام mTLS للمصادقة المتبادلة: كلا العميل والخادم يتحققان من هوية بعضهما بشهادات رقمية. العميل يقدم شهادته للخادم والعكس.

حالات الاستخدام لـ mTLS

- Service-to-service في الخدمات المصغرة: حماية التواصل بين الخدمات الموثوقة فقط.
- APIs داخلية: حماية واجهات البرمجة الداخلية عالية الأمان.
- أنظمة مالية/صحية: مصادقة قوية للطرفين في الصناعات الحساسة.
- يتطلب mTLS تكوين الخادم لطلب وتحقق شهادات العميل. في Spring Boot، يُكوّن للمصادقة القوية للعملاء.



Security Headers: Defense in Depth

يمكن تخصيصها في Spring Security عبر headers () لتناسب التطبيق:

1. Content-Security-Policy (CSP): تحدد المصادر الموثوقة للمحتوى (scripts, images, styles, fonts). تمنع XSS وحقق البيانات. مثال: 'default-src' 'self' يقصر على النطاق نفسه.
2. X-Content-Type-Options: nosniff: تمنع MIME sniffing؛ تجبر المتصفح على استخدام Content-Type المحدد فقط، مما يحمي من تنفيذ ملفات خاطئة كـ (XSS) JavaScript.
3. X-Frame-Options: DENY: تمنع عرض الموقع في frames/iframes على مواقع أخرى، لحماية من Clickjacking عبر إطارات شفافة.
4. X-XSS-Protection: 1; mode=block: تفعل مرشح XSS المدمج في المتصفحات؛ حماية إضافية للمتصفحات غير الداعمة الكاملة لـ CSP.
5. Referrer-Policy: strict-origin-when-cross-origin: تتحكم في رأس Referrer: ترسل Origin فقط عند cross-origin، والمسار الكامل داخل النطاق، لحماية الخصوصية.
6. Permissions-Policy: تتحكم في ميزات المتصفح مثل microphone, geolocation=() لمنع الوصول غير المرغوب، تعزيز الخصوصية.

Advanced Auth & Authorization: Spring Boot + JWT

أولاً: عملية التسجيل

نقوم بإنشاء دورة كاملة لإدارة المستخدمين.
أولاً، نحتاج إلى كيانات (Entities)
تمثل بيانات المستخدمين في قاعدة البيانات.

1. كيان المستخدم (UserEntity)

نستخدم JPA لإنشاء كيان يمثل بيانات المستخدمين في قاعدة البيانات. نستخدم Lombok لتقليل الكود المتكرر.

يستخدم Hibernate لتمثيل بيانات قاعدة البيانات كـ كيانات برمجية (Entities) تسهل التعامل مع البيانات.

```
@Data
@Entity
@Table(name = "users")
public class UserEntity implements UserDetails {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true, nullable = false)
    private String email;
    @Column(nullable = false)
    private String password;
    private String name;
    private LocalDateTime createdAt = LocalDateTime.now();
    // ميثودات واجهة UserDetails لربط الكيان بنظام أمن Spring
    @Override
    public Collection<? extends GrantedAuthority>
    getAuthorities() {
        return Collections.singletonList(new
        SimpleGrantedAuthority("ROLE_USER"));
    }
    @Override
    public String getUsername() { return email; }
    // ... ميثودات أخرى للتحقق من حالة الحساب
}
```

ملحظة أمنية: يسمح Spring Security بتنفيذ واجهة UserDetails مباشرة بالكيان لتحويله إلى كائن أمني.

2. كائن نقل البيانات (SignupDto)

نستخدم DTO (Client) لاستقبال البيانات من العميل قبل تطبيق قواعد التحقق (Validation)، ثم منع هجمات Over-Posting قبل وصول البيانات إلى طبقة الخدمة مباشرة.

```
@Data
public class SignupDto {
    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;
    @NotBlank(message = "Password is required")
    @Size(min = 6, message = "Password must be at least 6 characters")
    private String password;
    @NotBlank(message = "Name is required")
    private String name;
}
```

3. خدمة المصادقة (AuthService – Signup)

منطق العمل (Business Logic): يُستخدم عند الحاجة لمعالجة البيانات قبل حفظها، حيث يكمن "منطق العمل"، ونُشر كلمة المرور لحماية بيانات المستخدمين في حال اختراق قاعدة البيانات (Data Breach).

```
public UserDto signUp(SignupDto signupDto) {
    1 // التحقق من عدم وجود البريد مسبقاً
    Optional<UserEntity> user = userRepo.findByEmail(signupDto.getEmail());
    if(user.isPresent()) throw new BadCredentialsException("User already exists");
    2 // تحويل الـ DTO إلى Entity باستخدام ModelMapper
    UserEntity mappedUser = modelMapper.map(signupDto, UserEntity.class);
    3 // تشفير كلمة المرور (أهم خطوة أمنية)
    mappedUser.setPassword(passwordEncoder.encode(mappedUser.getPassword()));
    4 // الحفظ في قاعدة البيانات
    UserEntity savedUser = userRepo.save(mappedUser);
    return modelMapper.map(savedUser, UserDto.class);
}
```

ثانياً: عملية تسجيل الدخول وإنشاء JWT (Login & JWT)

في هذه العملية، نتحقق من صحة بيانات المستخدم. إذا كانت صحيحة، نُنشئ رمز JWT (Stateless)، بدلاً من إنشاء جلسات (Sessions) تقليدية.

1. أداة إنشاء JWT (JwtUtil)

هذا الكلاس هو المسؤول عن توليد الرموز (Token) والتحقق منها

ما هو الـ JWT؟

هو رمز مشفر رقمياً (Claims) يحتوي على معلومات المستخدم لضمان عدم التلاعب به.

لماذا نستخدمه؟

لجعل النظام Stateless، مما يسهل توسيع النظام وقدرته على التوزيع على عدة خوادم.

```
@Component
public class JwtUtil {
    @Value("${jwt.secret}")
    private String secret; // مفتاح سري للتوقيع الرقمي

    // توليد الرمز بناءً على بيانات المستخدم
    public String generateToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        return Jwts.builder()
            .setClaims(claims)
            .setSubject(userDetails.getUsername())
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() +
                86400000))
            // صالح لـ 24 ساعة /
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    // استخراج اسم المستخدم من الرمز للتحقق منه لاحقاً
    public String extractUsername(String token) { ... }
}
```



2. مرشح المصادقة بـ JWT (JwtAuthenticationFilter)

مرشح مصادقة JWT:

- دوره: يعمل "حارسًا" لكل طلب HTTP وارد.
- متى يعمل: قبل وصول الطلب إلى الـ Controller.
- لماذا: للتأكد أن المستخدم موثوق ولديه رمز صالح للوصول للموارد المحمية.

@Component

public class JwtAuthenticationFilter extends OncePerRequestFilter {

@Override

protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain) throws ServletException, IOException {

final String authHeader = request.getHeader("Authorization");

// 1. Bearer التأكد من وجود الرمز في الترويسة ويبدأ بـ

if (authHeader == null || !authHeader.startsWith("Bearer ")) {
filterChain.doFilter(request, response);
return;
}

// 2. استخراج الرمز والتحقق من صحته

final String jwt = authHeader.substring(7);

final String userEmail = jwtUtil.extractUsername(jwt);

// 3. Spring إذا كان الرمز صحيحًا، نضعه في "سياق الأمان"

if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
UserDetails userDetails = userDetailsService.loadUserByUsername(userEmail);
if (jwtUtil.validateToken(jwt, userDetails)) {

// تفعيل المصادقة لهذا الطلب فقط

UsernamePasswordAuthenticationToken authToken =

new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());

authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

SecurityContextHolder.getContext().setAuthentication(authToken);
}}
}filterChain.doFilter(request, response);
}

}

ثالثاً: إعدادات الأمن (SecurityConfig)

هنا نجمع كل ما سبق في تكوين واحد شامل.

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf().disable() // لأننا نستخدم CSRF تعطيل
            .authorizeHttpRequests(auth -> auth
                .antMatchers("/auth/**").permitAll() // السماح للجميع بالتسجيل والدخول
                .anyRequest().authenticated() // طلب المصادقة لأي مسار آخر
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS) // لا نريد جلسات في الخادم
            )
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
  
```

الممارسات الأمنية الأساسية (Key Security Practices)

الممارسة	الفائدة
تشفير كلمات المرور (BCrypt)	حماية كلمات المرور من السرقة المباشرة
استخدام DTOs	فصل بيانات العرض عن بيانات قاعدة البيانات ومنع هجمات الحقن
Statelessness JWT	تحسين الأداء وتسهيل التوسع دون الحاجة لتخزين الجلسات في الخادم
التحقق (Validation)	ضمان جودة البيانات المدخلة ومنع الأخطاء البرمجية
Transactional@	ضمان سلامة البيانات (إما أن تنجح العملية بالكامل أو تفشل بالكامل)

THE END